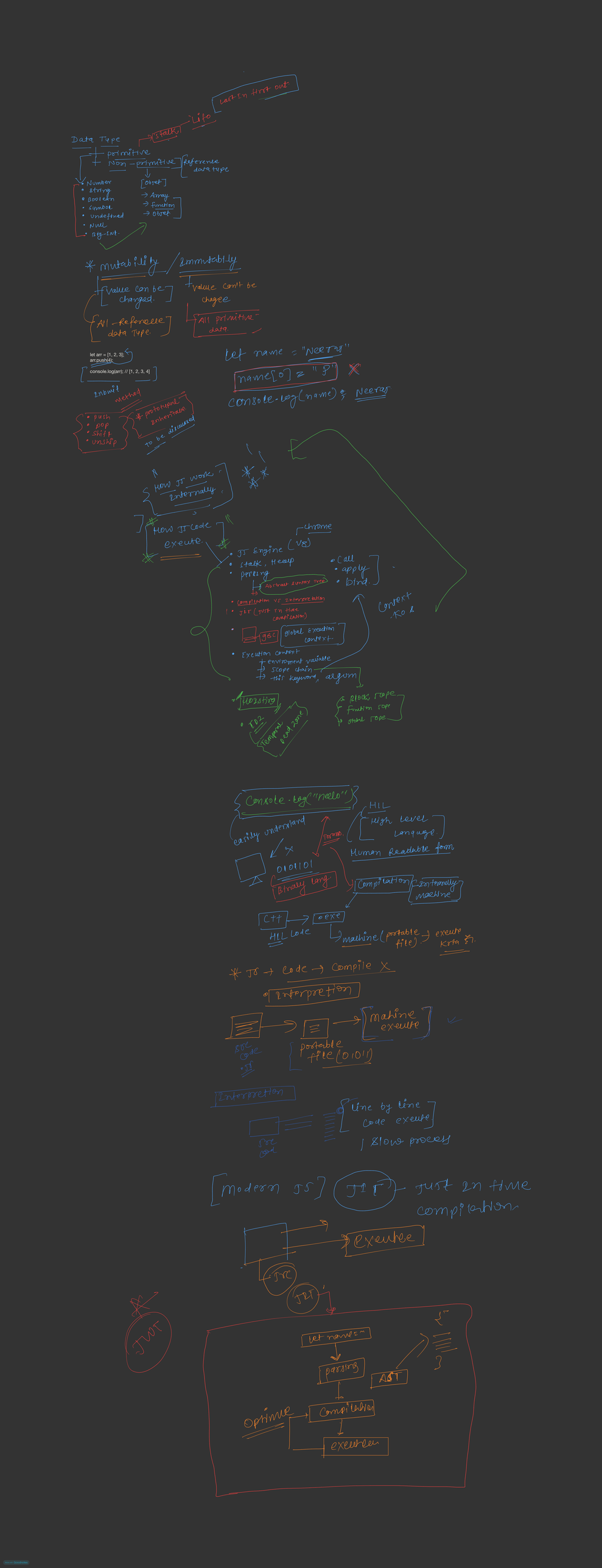

Data Type

- primitive → stack → Lifo → Last In first out
- Non-primitive → Reference data type

Number
- string
- Boolean
- Symbol
- Undefined
- Null
- Big-Int

[Object]
→ Array
→ function
→ Object

* mutability / Immutability

- value can be changed. → All-Reference data Type.
- value can't be chagee → All primitive data.

```
let arr = [1, 2, 3];
arr.push(4);
console.log(arr); // [1, 2, 3, 4]
```

Inbuilt method

- push
- pop
- shift
- unship

* prototypal inheritance to be discussed

let name = "Neeras"

name[0] = "P" ✗

console.log(name); Neeras

How JS work internally

How JS code execute

- JS Engine (V8) — chrome
- stack, Heap
- parsing
  - Abstract Syntax Tree
- Compilation vs Interpretation
- JIT (Just In time compilation)
- GEC — global Execution context
- Execution context
  - environment variable
  - scope chain
  - this keyword, argum

- call
- apply
- bind

Context KOs

- Hoisting
- TDZ — Temporal Dead zone

- Block scope
- function scope
- global scope

Console.log("hello")

easily understand

0101101 — Binary lang

Trans.

HLL — High Level Language — Human Readable form

Compilation — Internally machine

C++ → .exe

HLL Code

machine (portable file) → execute Krta ☆

* JS → code → Compile ✗

Interpretation

src code .JS → portable file (0101) → machine execute

Interpretation

src code

line by line code execute

slow process

[Modern JS] JIT — Just In time compilation

Executee

JRC

JIT

JWT

let name = ~

parsing

AST

Optimize

Compilation

executee